

HTML Ingestion Setup -> Ingestion Webhook

Why I built this

IngestionWebhook (Agent 1) exists to receive real checkout events from an e-commerce storefront. This assignment only asked me to build the **agent**, not a storefront — so there was never going to be a real system POSTing events to it. To build and verify Agent 1 (and, downstream, Agent 2's triage logic), I needed **something** to send it realistic checkout events on demand. That's the entire reason this file exists: it's a developer testing convenience that sits in front of the pipeline, not a component of the pipeline itself.

How it started: curl commands

Before any HTML existed, testing was done with plain **curl** from the command line — one separate command per scenario, run one at a time against the webhook, to exercise every branch of the decision logic:

1. Low value / isolated event (expected: LOG, low severity)

```
curl -X POST "<test-or-production-webhook-url>" \
-H "Content-Type: application/json" \
-d '{"checkout_step": "ADDRESS", "cart_value": 45, "device": "MOBILE"}'
```

2. Payment failure (expected: ALERT, HIGH — PAYMENT step alone triggers HIGH)

```
curl -X POST "<test-or-production-webhook-url>" \
-H "Content-Type: application/json" \
-d '{"checkout_step": "PAYMENT", "cart_value": 899, "device": "DESKTOP"}'
```

3. High cart value (expected: ALERT, HIGH — value threshold triggers regardless of step)

```
curl -X POST "<test-or-production-webhook-url>" \
-H "Content-Type: application/json" \
-d '{"checkout_step": "SHIPPING", "cart_value": 4599, "device": "DESKTOP"}'
```

4. Mobile failure pattern (expected: ALERT, HIGH once repeated — send this one 2-3 times to simulate a pattern)

```
curl -X POST "<test-or-production-webhook-url>" \
-H "Content-Type: application/json" \
-d '{"checkout_step": "SHIPPING", "cart_value": 120, "device": "MOBILE"}'
```

5. Invalid payload (expected: 400 error — tests the IF node's validation branch)

```
curl -X POST "<test-or-production-webhook-url>" \
-H "Content-Type: application/json" \
-d '{"checkout_step": "UNKNOWN_STEP", "cart_value": "not-a-number", "device": "MOBILE"}'
```

Each of these had to be typed or pasted separately, editing the **-d** payload by hand every time a new scenario was needed — this repetition is exactly what motivated moving to the HTML tool next.

Where these were run: in the n8n canvas, click **Listen for test event** on the **IngestionWebhook** node — this arms n8n's **Test URL** to accept exactly one incoming call. The **curl** command was then run from a terminal, pointed at that Test URL (shown directly in the node, e.g. **.../webhook-test/checkout-event**). Once the workflow was **Activated**, the same commands worked against the **Production URL** instead, for continuous (not one-shot) listening.

Why I moved to build a small page

`curl` worked, but retyping/editing raw JSON for every test run — especially to hit five different scenarios repeatedly while debugging node-by-node — was slow and error-prone. After the first round of `curl` testing, I moved to build a small page that does the same POST with a form and one-tap presets instead of hand-written JSON.

The prompt that created it

This grew directly out of the workflow: rather than continuing to copy and edit `curl` commands by hand for every test run, I moved to build a clickable form instead — a small, self-contained HTML page that could fire the same test scenarios with a click. Refined into a clean, reusable form (rather than the exact back-and-forth phrasing used in the moment), the prompt was:

"Build a small, self-contained HTML page (no server, no build step — just open it in a browser) for testing an n8n webhook that ingests checkout events. It should let me: paste in the webhook URL; fill in checkout_step, cart_value, and device manually, or tap one-click presets for standard test scenarios (a normal low-value event, a payment failure, a high cart value, a repeated mobile-failure pattern, and an invalid payload); preview the exact JSON before sending, for any preset that can't be represented cleanly by the form fields (e.g. a non-numeric value); and log each request's response inline, so I don't have to switch over to n8n's execution panel to see whether it worked."

Where it was created and where it runs — two different places, worth distinguishing:

- **Created:** directly inside this Claude conversation, using Claude's built-in file-creation capability (no separate IDE, no external coding tool). The HTML/CSS/JS was written and saved as a file (`checkout_event_sender.html`) in that same session, then made available to download.
- **Runs:** entirely client-side, in an ordinary web browser, completely separate from Claude — once downloaded, it's a static file with no server and no dependency on Claude at all. Opening it in a browser and pointing it at the n8n webhook URL is all that's needed; Claude is not involved at runtime.

What the tool does

A single self-contained HTML file (no server, no build step — just open it in a browser) that replaces all five `curl` commands above with one reusable page:

curl scenario	HTML preset
Low value / isolated event	"Low value / Address" chip
Payment failure	"Payment failure" chip
High cart value	"High cart value" chip
Mobile failure pattern	"Mobile pattern" chip
Invalid payload	"Invalid payload" chip

On top of reproducing those five fixed scenarios as one-tap presets, it adds:

- **Webhook URL field** — paste in either the Test URL or Production URL once; nothing else needs to change to switch between them (with `curl`, the URL had to be re-pasted into every single command).

- **Event details form** — checkout step, device, cart value — editable directly, for scenarios beyond the original five (e.g. testing a boundary value like exactly **1500**, which none of the five presets cover).
- **Source tag field** — added after testing revealed a gap (see below).
- **Send event button** — fires the POST and logs the response (status code + body) inline, so you can see success/failure without switching over to n8n's execution panel or a separate terminal window.

How it helped with developer testing convenience

- **No retyping JSON per test** — a click sets the form, a click sends it.
- **Consistent behavior across every preset** — this wasn't true initially (see "Bugs found," below); after fixing it, every preset follows the same preview-then-send flow, so nothing fires unexpectedly.
- **Immediate visual feedback** — the in-page log shows exactly what was sent and what came back, without needing to open n8n at all to sanity-check a request.
- **Reusable across the whole session** — one static file, works against Test or Production URLs interchangeably, no reconfiguration needed as the workflow evolved.
- **A payload preview for cases the form can't represent** — the Invalid payload preset uses a non-numeric **cart_value**, which can't be typed into a **type="number"** field. Rather than fabricate that behind the scenes, the tool shows the exact raw JSON about to be sent in a clearly marked "override" box before you click Send.

Bugs this tool surfaced in itself (and the fixes)

- **Original bug:** the Invalid payload preset auto-sent immediately on click, while every other preset just populated the form and waited for **Send event**. This was inconsistent and misleading — it looked like a bypass rather than a deliberate action.

Fix: all presets now behave identically. Representable presets (step in **PAYMENT/ADDRESS/SHIPPING**, numeric cart value) fill the form; non-representable ones (like Invalid payload) show a raw-JSON preview box and wait for an explicit **Send event** click — same as everything else.

- **Traceability gap:** none of the preset labels ("Payment failure," "Invalid payload," etc.) were ever transmitted anywhere — Agent 1 only ever saw the four raw event fields, so there was no way to tell, later, which sheet row came from which test scenario.

Fix: added a **source_tag** field, auto-filled per preset (e.g. **preset_payment_failure**) or defaulting to **manual_test**, sent alongside the real fields in every POST.

What changes this required in the n8n workflow

Testing with this tool — specifically the Invalid payload and manually-edited cases — surfaced real gaps in Agent 1 that wouldn't have shown up from testing with only well-formed data:

- **`Edit Fields` node** — originally crashed outright on a non-numeric **cart_value** (a hard type-conversion error), instead of failing validation gracefully. Rewritten to **isNaN(Number(\$json.body.cart_value)) ? -1 : Number(\$json.body.cart_value)**, so a bad value becomes **-1** and is correctly caught by the IF node next, rather than halting the execution.
- **`Edit Fields` node** — **checkout_step** and **device** were made null-safe (**?? ''**) so a payload missing a field entirely fails validation instead of crashing.

- **`Edit Fields` node** — added a `source_tag` mapping (`$json.body.source_tag ?? 'live'`) to carry the new traceability field through to the sheet.
- **`If` node** — added a third condition validating `device` is `MOBILE/DESKTOP`; the original design only checked `checkout_step` and `cart_value`.
- **`Respond to Webhook` (false branch)** — changed from one fixed error string to an expression that names the specific field that failed validation, since testing multiple invalid scenarios back-to-back made a generic message unhelpful for confirming *which* check actually caught the bad input.
- **`IngestionWebhook` node** — the **Respond** setting had to be switched from the default `onReceived` to `Using 'Respond to Webhook' Node`; otherwise n8n auto-replies the instant the request arrives, and neither the success (200) nor error (400) `Respond to Webhook` nodes downstream ever actually get to send their response — this was only caught because the HTML tool's log showed a generic response instead of the expected 400 on invalid payloads.

None of these were things I'd planned upfront — each came directly out of actually exercising the webhook with realistic (and deliberately broken) input through this tool, rather than only checking the happy path.

Final artifacts (as shared)

- **`checkout\_event\_sender.html`** — the test console described above.
- **`02-n8nIncidentDetectionReasoning.json`** — the exported n8n workflow.

One thing worth flagging on the exported JSON: it still includes the `Google Gemini Chat Model` and `AI Agent` nodes (the `Assemble the Command Row` code references `JSON.parse($json.output)`, i.e. an LLM output). That reflects the version **before** the later no-LLM simplification I did — where `Aggregate Batch`'s precomputed `triggered_rules_text` is templated directly into `reason`, with no model call at all. If you'd like, I can mark up exactly which nodes to remove and which one line in `Assemble the Command Row` to change to bring this exported JSON in line with the no-LLM version.